

INSTITUTO TECNOLÓGICO SUPERIOR DE POZA RICA



REPORTE DE AVANCE DE PROYECTO 1 – UNO

INTEGRANTES DEL EQUIPO:

- **FERNANDA PARADA CASTELLANOS**
MATRICULA: 23ISIC067
- **MILDRED NAHOMI CORTEZ VILLANUEVA**
MATRICULA: 23ISIC064
- **FLOR ITZEL FRANCO LUNA**
MATRICULA: 23ISIC082
- **CARMEN XIMENA ZAMORA RODRÍGUEZ**
MATRICULA: 23ISIC070

MATERIA:

LENGUAJE DE INTERFAZ

FECHA: 27/03/2026

REPORTE DE AVANCE DEL PROYECTO

Implementación del Juego UNO con Lenguaje Ensamblador y C++

1. Resumen de Actividades Realizadas

Durante este periodo se trabajó en el desarrollo inicial del proyecto, enfocado en la creación de un prototipo funcional del juego UNO mediante la integración de C++ y lenguaje ensamblador. Se configuró el entorno de desarrollo en Ubuntu y WSL, instalando NASM y G++ para la compilación de código ensamblador y C++. Se diseñó la estructura del proyecto con los archivos main.cpp para la interfaz y uno.asm para la lógica del juego. Se implementó la generación de cartas, la validación de jugadas y el control de turnos en lenguaje ensamblador, y se realizaron pruebas iniciales desde la consola para verificar la correcta integración entre C++ y ASM. Con este avance se establecieron las bases para integrar posteriormente la interfaz gráfica utilizando SFML.

2. Actividades realizadas en el avance a entregar para la fecha 27 de marzo:

- Configuración del entorno con WSL
- Instalación de NASM y G++
- Diseño de la estructura del proyecto
- Desarrollo de main.cpp (interfaz)
- Desarrollo de uno.asm (lógica)
- Integración entre C++ y ensamblador
- Pruebas iniciales del sistema

3. Herramientas y Tecnologías Utilizadas

Lenguajes:

- C++
- Ensamblador (NASM)

Herramientas:

- WSL
- NASM
- G++
- Terminal Linux

Estándares:

- Programación modular
- Integración de alto y bajo nivel

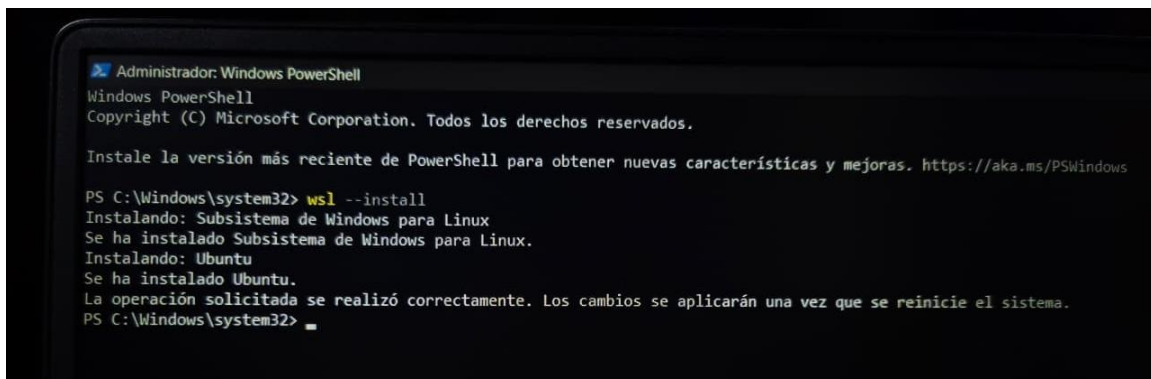
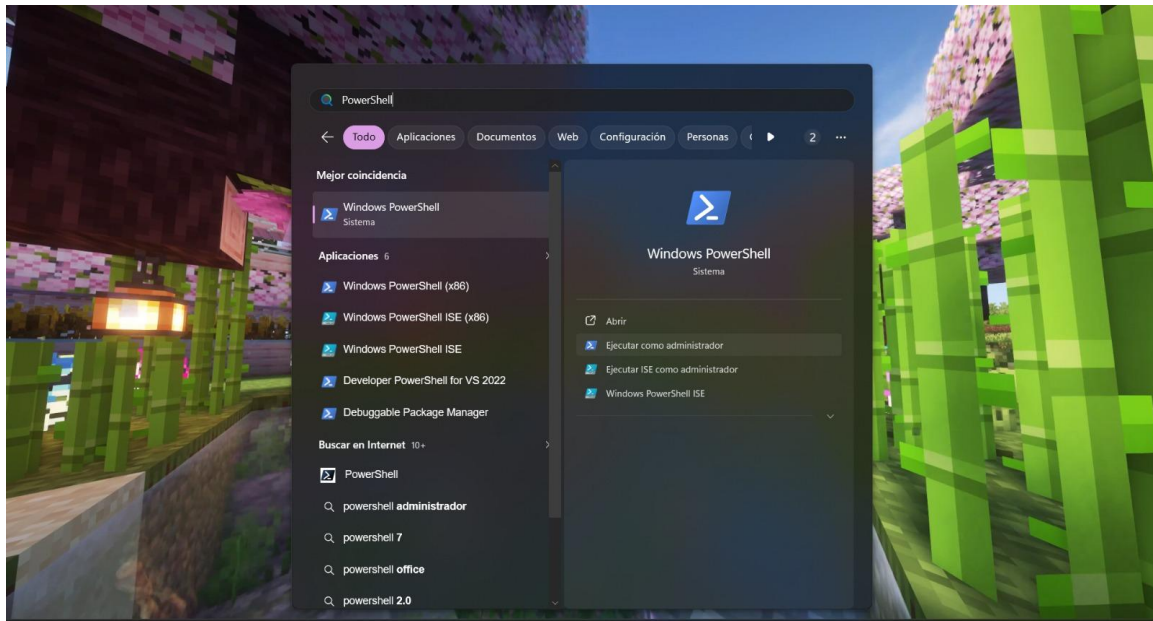
Evidencia Técnica

1- Configuración del entorno con WSL

Instalación de WSL (Windows Subsystem for Linux).

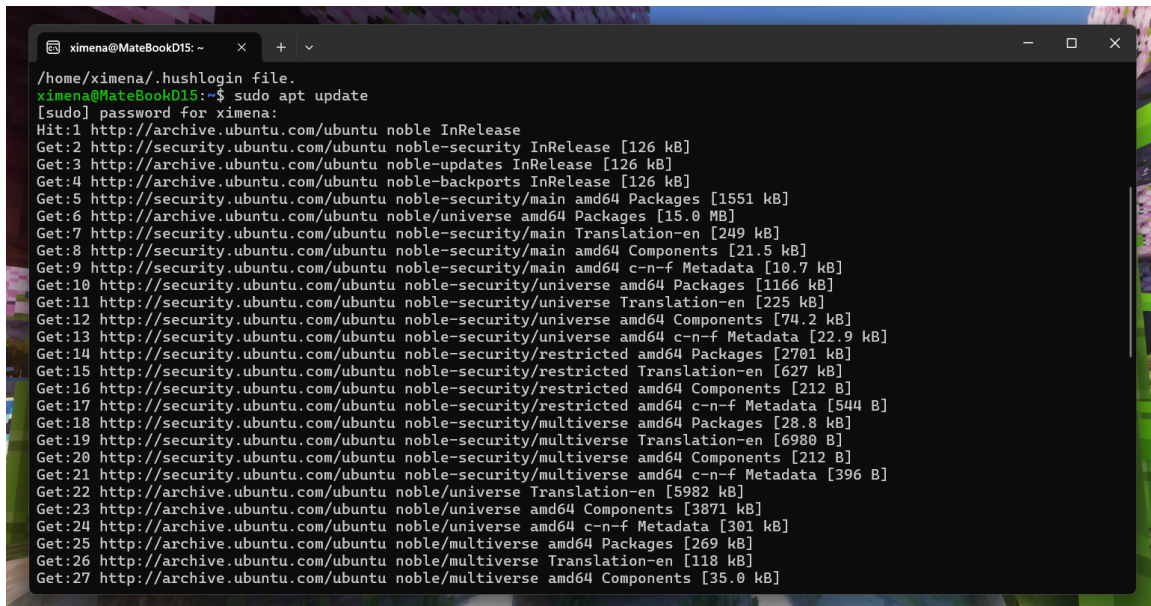
Es una herramienta que permite ejecutar Ubuntu u otra distribución Linux directamente en Windows, sin necesidad de una máquina virtual completa. Sirve para compilar y ejecutar tu código en Linux.

Se habilitó el Subsistema de Windows para Linux (WSL) y se instaló la distribución Ubuntu para poder compilar y ejecutar código en lenguaje ensamblador y C++.



2. Instalación de NASM y G++

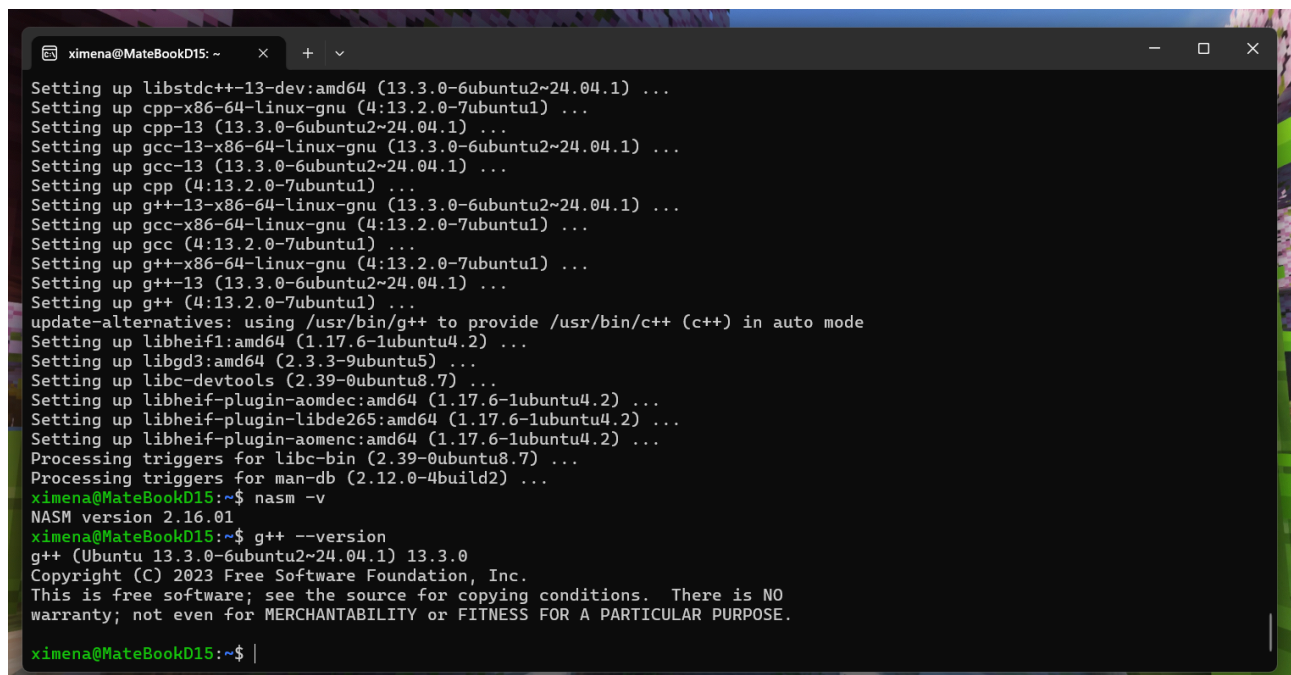
Se actualizaron los paquetes del sistema.

A terminal window titled 'ximena@MateBookD15: ~' showing the output of the 'sudo apt update' command. The output lists various Ubuntu repositories and the packages they provide, including InRelease files, Packages, Translation-en files, and Components. The list ends with 'Get:27 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]'.

```
/home/ximena/.hushlogin file.
ximena@MateBookD15:~$ sudo apt update
[sudo] password for ximena:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1551 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [249 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.7 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1166 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [225 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [22.9 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2701 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [627 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [544 B]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.8 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6980 B]
Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:21 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [396 B]
Get:22 http://archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:24 http://archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:25 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:26 http://archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:27 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
```

Después se instalaron las herramientas necesarias que vamos a ocupar, en este caso son, NASM y G++.

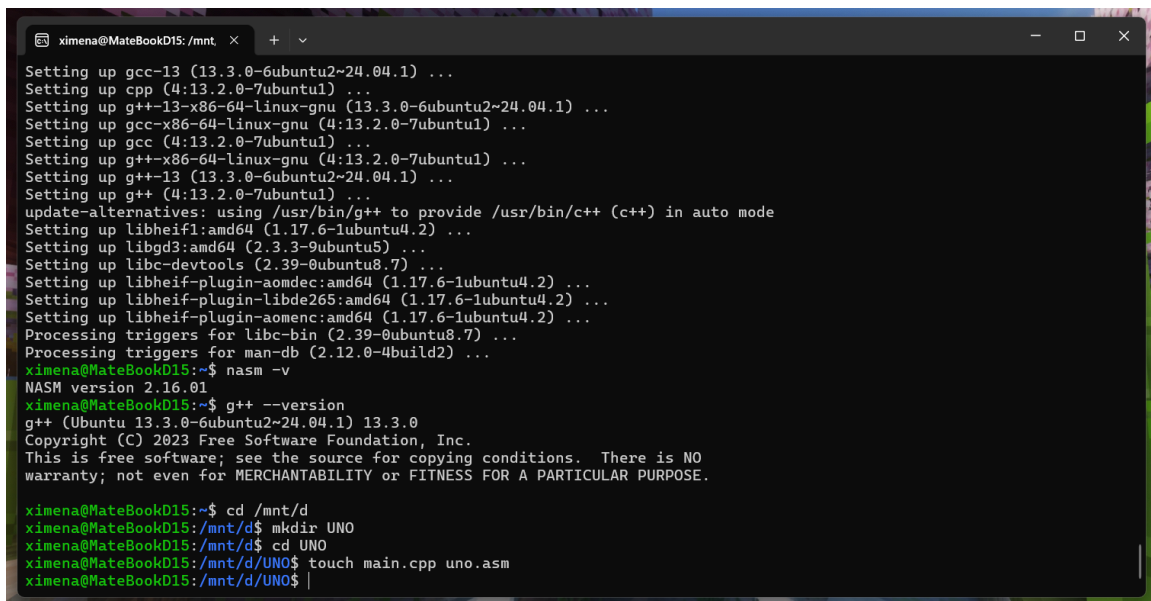
- NASM es el ensamblador
- G++ es el compilador C++

A terminal window titled 'ximena@MateBookD15: ~' showing the output of the 'sudo apt install libstdc++-13-dev gcc-13 g++-13' command. The output lists the packages being installed and their versions, followed by the installation of the g++ compiler. The terminal shows the g++ version as 13.3.0 and the NASM version as 2.16.01.

```
Setting up libstdc++-13-dev:amd64 (13.3.0-6ubuntu2~24.04.1) ...
Setting up cpp-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up cpp-13 (13.3.0-6ubuntu2~24.04.1) ...
Setting up gcc-13-x86-64-linux-gnu (13.3.0-6ubuntu2~24.04.1) ...
Setting up gcc-13 (13.3.0-6ubuntu2~24.04.1) ...
Setting up cpp (4:13.2.0-7ubuntu1) ...
Setting up g++-13-x86-64-linux-gnu (13.3.0-6ubuntu2~24.04.1) ...
Setting up gcc-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up gcc (4:13.2.0-7ubuntu1) ...
Setting up g++-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up g++-13 (13.3.0-6ubuntu2~24.04.1) ...
Setting up g++ (4:13.2.0-7ubuntu1) ...
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mode
Setting up libheif1:amd64 (1.17.6-1ubuntu4.2) ...
Setting up libgd3:amd64 (2.3.3-9ubuntu5) ...
Setting up libc-devtools (2.39-0ubuntu8.7) ...
Setting up libheif-plugin-aomdec:amd64 (1.17.6-1ubuntu4.2) ...
Setting up libheif-plugin-libde265:amd64 (1.17.6-1ubuntu4.2) ...
Setting up libheif-plugin-aomenc:amd64 (1.17.6-1ubuntu4.2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for man-db (2.12.0-4build2) ...
ximena@MateBookD15:~$ nasm -v
NASM version 2.16.01
ximena@MateBookD15:~$ g++ --version
g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
ximena@MateBookD15:~$
```

3. Diseño de la estructura del proyecto

Se diseñó la estructura del proyecto dividiendo las responsabilidades en dos archivos principales: *uno.asm* para la implementación de la lógica del juego en lenguaje ensamblador y *main.cpp* para la interfaz de usuario en C++. La comunicación entre ambos se realiza mediante funciones exportadas desde ASM y llamadas desde C++ usando `extern "C"`. Se organizó la carpeta del proyecto para incluir los archivos fuente, posibles headers y recursos gráficos como fuentes, de manera que se facilite la compilación y posterior integración de la interfaz gráfica con SFML.



```
ximena@MateBookD15: /mnt. x + v
Setting up gcc-13 (13.3.0-6ubuntu2~24.04.1) ...
Setting up cpp (4:13.2.0-7ubuntu1) ...
Setting up g++-13-x86-64-linux-gnu (13.3.0-6ubuntu2~24.04.1) ...
Setting up gcc-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up gcc (4:13.2.0-7ubuntu1) ...
Setting up g++-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up g++-13 (13.3.0-6ubuntu2~24.04.1) ...
Setting up g++ (4:13.2.0-7ubuntu1) ...
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mode
Setting up libheif1:amd64 (1.17.6-1ubuntu4.2) ...
Setting up libgd3:amd64 (2.3.3-9ubuntu5) ...
Setting up libc-devtools (2.39-0ubuntu8.7) ...
Setting up libheif-plugin-aomdec:amd64 (1.17.6-1ubuntu4.2) ...
Setting up libheif-plugin-libde265:amd64 (1.17.6-1ubuntu4.2) ...
Setting up libheif-plugin-aomenc:amd64 (1.17.6-1ubuntu4.2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for man-db (2.12.0-4build2) ...
ximena@MateBookD15:~$ nasm -v
NASM version 2.16.01
ximena@MateBookD15:~$ g++ --version
g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

ximena@MateBookD15:~$ cd /mnt/d
ximena@MateBookD15:/mnt/d$ mkdir UNO
ximena@MateBookD15:/mnt/d$ cd UNO
ximena@MateBookD15:/mnt/d/UNO$ touch main.cpp uno.asm
ximena@MateBookD15:/mnt/d/UNO$ |
```

4. Desarrollo de *main.cpp* (interfaz)

El archivo *main.cpp* fue desarrollado como la interfaz principal del juego, inicialmente en modo consola. Su función principal es interactuar con el jugador, mostrando las cartas en mano, la carta actual en la mesa y las opciones disponibles (jugar carta, robar, declarar UNO o salir). Además, gestiona los turnos de los demás jugadores controlados por IA y realiza llamadas a las funciones implementadas en *uno.asm* para validar las jugadas y aplicar las reglas del juego. Esta estructura permite que, en fases posteriores, se pueda integrar una interfaz gráfica utilizando SFML sin modificar la lógica de las reglas del juego.

```
ximena@MateBookD15: /mnt  x + v
GNU nano 7.2 main.cpp
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>

using namespace std;

extern "C" int carta_valida(int jugador, int mesa);

#define ROJO "\033[31m"
#define AZUL "\033[34m"
#define VERDE "\033[32m"
#define AMARILLO "\033[33m"
#define RESET "\033[0m"

string nombres[] = {
    "0", "1", "2", "3", "4", "5", "6", "7", "8", "9",
    "+2", "Pierde turno", "Reversa", "Comodin", "+4"
};

string coloresTexto[] = {"", ROJO, AZUL, VERDE, AMARILLO};
string nombreColor[] = {"", "Rojo", "Azul", "Verde", "Amarillo"};

struct Carta {
    int color;
    int valor;
};

void mostrarCarta(Carta c) {
    if (c.valor >= 13) {
        cout << nombres[c.valor];
        if (c.color != 0)
            cout << " (" << nombreColor[c.color] << ")";
    } else {
        cout << coloresTexto[c.color] << nombres[c.valor] << RESET;
    }
}

[ Read 248 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^M-U Undo     ^M-A Set Mark ^M-J To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line ^M-E Redo     ^M-6 Copy     ^_ Where Was
```

```
ximena@MateBookD15: /mnt  x + v
GNU nano 7.2 main.cpp

Carta generarCarta() {
    Carta c;
    c.valor = rand() % 15;
    c.color = (c.valor >= 13) ? 0 : rand() % 4 + 1;
    return c;
}

bool validar(Carta j, Carta m) {
    if (j.valor >= 13) return true;
    if (j.color == m.color) return true;
    if (j.valor == m.valor) return true;

    int cj = j.color * 10 + j.valor;
    int cm = m.color * 10 + m.valor;

    return false;
}

int elegirColor() {
    int c;
    cout << "Elige color:\n1.Rojo 2.Azul 3.Verde 4.Amarillo\n";
    cin >> c;
    return c;
}

int main() {
    srand(time(0));

    int jugadores;
    cout << "Numero de jugadores (incluyendote): ";
    cin >> jugadores;

    vector<vector<Carta>> manos(jugadores);

    for (int i = 0; i < jugadores; i++)
        for (int j = 0; j < 5; j++)

Cancelled
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^M-U Undo     ^M-A Set Mark ^M-J To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line ^M-E Redo     ^M-6 Copy     ^_ Where Was
```

```
ximena@MateBookD15: /mnt  ×  +  ▾
GNU nano 7.2 main.cpp

    manos[i].push_back(generarCarta());

    Carta mesa = generarCarta();

    // FIX inicio comodin/+4
    if (mesa.valor == 13 || mesa.valor == 14) {
        mesa.color = rand() % 4 + 1;
        cout << "Carta inicial: " << nombres[mesa.valor] << endl;
        cout << "Color asignado: " << nombreColor[mesa.color] << endl;
    }

    int turno = 0, direccion = 1;
    int robar = 0;
    bool saltar = false;

    while (true) {
        cout << "\n=====n";
        cout << "Turno jugador " << turno + 1 << endl;

        // EFECTOS
        if (robar > 0) {
            cout << "Jugador " << turno + 1 << " roba " << robar << " cartas!\n";
            for (int i = 0; i < robar; i++)
                manos[turno].push_back(generarCarta());
            robar = 0;
            turno = (turno + direccion + jugadores) % jugadores;
            continue;
        }

        if (saltar) {
            cout << "Jugador " << turno + 1 << " pierde turno!\n";
            saltar = false;
            turno = (turno + direccion + jugadores) % jugadores;
            continue;
        }

        cout << "Carta en mesa: ";

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-J To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-6 Copy      ^Q Where Was
```

```
    if (turno == 0) {

        bool dijoUNO = false;

        while (true) {

            cout << "\nTus cartas:\n";
            for (int i = 0; i < manos[0].size(); i++) {
                cout << i << " ";
                mostrarCarta(manos[0][i]);
                cout << endl;
            }

            int op;
            cout << "\nOpciones:\n-1 Robar\n-2 Salir\n-3 UNO\n";
            cin >> op;

            if (op == -2) return 0;

            if (op == -3) {
                if (manos[0].size() == 2) {
                    cout << "UNO!\n";
                    dijoUNO = true;
                }
                continue;
            }

            if (op == -1) {
                Carta nueva = generarCarta();
                manos[0].push_back(nueva);

                cout << "Robaste: ";
                mostrarCarta(nueva);
                cout << endl;

                continue;
            }
        }
    }

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-J To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-6 Copy      ^Q Where Was
```



```
ximena@MateBookD15:/mnt  ×  +  ▾
GNU nano 7.2 main.cpp
}

if (op >= 0 && op < manos[0].size()) {

    Carta c = manos[0][op];

    if (!validar(c, mesa)) {
        cout << "Movimiento invalido\n";
        continue;
    }

    int antes = manos[0].size();

    mesa = c;
    manos[0].erase(manos[0].begin() + op);

    if (antes == 2 && !dijoUNO) {
        cout << "No dijiste UNO! +2 cartas\n";
        manos[0].push_back(generarCarta());
        manos[0].push_back(generarCarta());
    }

    if (c.valor == 10) robar = 2;
    else if (c.valor == 11) saltar = true;
    else if (c.valor == 12) direccion *= -1;
    else if (c.valor == 13) mesa.color = elegirColor();
    else if (c.valor == 14) {
        mesa.color = elegirColor();
        robar = 4;
    }

    break;

}

cout << "Opcion invalida\n";

}

}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^I Execute    ^C Location   ^U Undo       ^M Set Mark   ^= To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/_ Go To Line ^E Redo       ^G Copy       ^_ Where Was
```

```
ximena@MateBookD15:/mnt  ×  +  ▾
GNU nano 7.2 main.cpp *
else {

    while (true) {

        int jugada = -1;

        for (int i = 0; i < manos[turno].size(); i++) {
            if (validar(manos[turno][i], mesa)) {
                jugada = i;
                break;
            }
        }

        if (jugada == -1) {
            Carta nueva = generarCarta();
            manos[turno].push_back(nueva);
            cout << "Jugador " << turno + 1 << " roba una carta\n";
        } else {

            int antes = manos[turno].size();

            if (antes == 2) {
                cout << "Jugador " << turno + 1 << " dice UNO!\n";
            }

            Carta c = manos[turno][jugada];

            cout << "Jugador " << turno + 1 << " juega: ";
            mostrarCarta(c);
            cout << endl;

            mesa = c;
            manos[turno].erase(manos[turno].begin() + jugada);

            if (c.valor == 10) robar = 2;
            else if (c.valor == 11) saltar = true;

        }

    }

}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^I Execute    ^C Location   ^U Undo       ^M Set Mark   ^= To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/_ Go To Line ^E Redo       ^G Copy       ^_ Where Was
```

```
        else if (c.valor == 12) direccion *= -1;
        else if (c.valor == 13 || c.valor == 14) {
            mesa.color = rand() % 4 + 1;
            if (c.valor == 14) robar = 4;
        }

        break;
    }

}

if (manos[turno].empty()) {
    cout << "\n Jugador " << turno + 1 << " gana!\n";
    break;
}

turno = (turno + direccion + jugadores) % jugadores;

}

return 0;

}
```


5. Desarrollo de uno.asm (lógica)

Dentro del archivo uno.asm, se desarrolló la función `carta_valida` para verificar si una carta seleccionada por el jugador es válida sobre la carta que está en la mesa. Cada carta se codifica como $\text{color} * 10 + \text{valor}$, y la función separa el color y valor utilizando divisiones enteras. Si el color o el valor coinciden con la carta en mesa, la función devuelve 1 indicando que la jugada es válida; de lo contrario, devuelve 0. Esta función es exportada para ser llamada desde `main.cpp`, permitiendo que la interfaz en C++ reciba la respuesta y actúe en consecuencia.



```
GNU nano 7.2 uno.asm
section .text
global carta_valida

carta_valida:
    mov rax, rdi
    mov rcx, 10
    xor rdx, rdx
    div rcx
    mov r8, rax
    mov r9, rdx

    mov rax, rsi
    xor rdx, rdx
    div rcx
    mov r10, rax
    mov r11, rdx

    cmp r8, r10
    je valido

    cmp r9, r11
    je valido

    mov rax, 0
    ret

valido:
    mov rax, 1
    ret
```

6. Integración entre C++ y ensamblador

Se implementó la integración entre C++ y ensamblador utilizando la declaración `extern "C"` en `main.cpp`, lo que permite llamar a funciones de `uno.asm` directamente desde el código C++ sin problemas de enlace. La interfaz en C++ envía los parámetros necesarios a las funciones ASM (por ejemplo, carta del jugador y carta en mesa) mediante registros y recibe los resultados en `rax`. Esta integración permite que la lógica del juego implementada en ensamblador se ejecute correctamente mientras la interfaz de usuario en C++ se encarga de mostrar la información, controlar turnos y recibir entradas del jugador.

7. Pruebas iniciales del sistema

Se realizaron pruebas iniciales del sistema para verificar el correcto funcionamiento de la integración entre C++ y ensamblador. Se comprobó que la función `carta_valida` retorna los resultados esperados en distintas combinaciones de cartas. Además, se validó que la interfaz en `main.cpp` muestra correctamente las cartas, permite al jugador realizar acciones y que la IA ejecuta sus turnos automáticamente. Todas las pruebas confirmaron que la comunicación entre la lógica en ensamblador y la interfaz en C++ funciona correctamente, cumpliendo con los requerimientos iniciales del proyecto.

```
ximena@MateBookD15: /mnt. x + v
ximena@MateBookD15:/mnt/d/UNO$ nano main.cpp
ximena@MateBookD15:/mnt/d/UNO$ ./juego
Numero de jugadores (incluyendote): 3
Carta inicial: +4
Color asignado: Rojo

=====
Turno jugador 1
Carta en mesa: +4 (Rojo)

Tus cartas:
0: 8
1: Pierde turno
2: Comodin
3: 6
4: 9

Opciones:
-1 Robar
-2 Salir
-3 UNO
3

=====
Turno jugador 2
Carta en mesa: 6
Jugador 2 roba una carta
Jugador 2 juega: 3

=====
Turno jugador 3
Carta en mesa: 3
Jugador 3 juega: Reversa

=====
Turno jugador 2
Carta en mesa: Reversa
Jugador 2 roba una carta
Jugador 2 roba una carta
Jugador 2 juega: 3
```

```
Jugador 2 juega: 2

=====
Turno jugador 3
Carta en mesa: 2
Jugador 3 roba una carta
Jugador 3 roba una carta
Jugador 3 juega: 2

=====
Turno jugador 1
Carta en mesa: 2

Tus cartas:
0: 8
1: 5

Opciones:
-1 Robar
-2 Salir
-3 UNO
-3
UNO!

Tus cartas:
0: 8
1: 5

Opciones:
-1 Robar
-2 Salir
-3 UNO
1

=====
Turno jugador 2
Carta en mesa: 5
Jugador 2 juega: 8
🎉 Jugador 2 gana!
ximena@MateBookD15:/mnt/d/UNO$ |
```

Problemas y Soluciones

Durante el desarrollo del proyecto se presentaron algunos problemas técnicos relacionados con la compilación, ejecución del programa y la integración entre C++ y lenguaje ensamblador. Asimismo, se identificaron dificultades en la lógica del sistema durante las primeras pruebas. Sin embargo, todos estos inconvenientes fueron analizados y solucionados correctamente, lo que permitió continuar con el desarrollo del proyecto de manera estable y funcional.

Planificación del Siguiente Hito

1. Diseño de la interfaz gráfica (parte visual) – 31 de marzo de 2026
2. Diseño de la interfaz gráfica (parte interactiva) – 15 de abril de 2026
3. Pruebas de la interfaz gráfica y evidencia visual – 29 de abril de 2026

Mejoras futuras propuestas

Para continuar con el desarrollo del proyecto, se proponen las siguientes mejoras:

- Implementar una interfaz gráfica utilizando SFML, para mejorar la experiencia del usuario.
- Mejorar la IA de los jugadores, para que tome decisiones más inteligentes.
- Incorporar animaciones y efectos visuales en la interfaz gráfica.
- Añadir opción de multijugador local o en red.



Aprender

Durante el desarrollo de este avance se lograron los siguientes aprendizajes:

- Comprensión de la integración entre lenguajes de alto nivel (C++) y bajo nivel (ensamblador).
- Manejo de herramientas de desarrollo en entorno Linux mediante WSL.
- Aplicación de programación modular para separar la lógica del sistema.
- Desarrollo de habilidades en depuración y solución de errores en distintos niveles del sistema.



Conclusión

Se logró integrar C++ con ensamblador, validando jugadas del juego UNO. El proyecto establece bases sólidas para continuar su desarrollo.

Conclusión General

El proyecto permitió comprender la programación de bajo nivel y su integración con lenguajes modernos. Se continuará mejorando hasta lograr un sistema completo.